

Sistema de Controle de Acesso Biométrico com Processamento Local e Defesa contra Ataques de Apresentação

Fernando de Melo Colli - 231026349
Faculdade de Ciências e Tecnologias em Engenharia (FCTE)
Universidade de Brasília

Resumo—Este trabalho apresenta o refinamento final de um sistema de controle de acesso residencial biométrico, embarcado integralmente em uma Raspberry Pi 3 Model B com processamento local (*Edge Computing*). No PC4, todo o software foi migrado de Python para C/C++, consolidando um sistema concorrente de dois processos que se comunicam por um *pipe* nomeado (FIFO): uma *pipeline* de visão (*visao*) e um núcleo de controle de tempo real (*sistema_core*). O núcleo passou a empregar recursos de tempo real do Linux — travamento de memória (*mlockall*), escalonamento *SCHED_FIFO*, afinidade de CPU e tarefa periódica com *clock_nanosleep* absoluto — garantindo latência determinística no acionamento da trava. O reconhecimento facial (LBPH implementado do zero, sem dependência do *opencv_contrib*) recebeu normalização de iluminação Tan-Triggs idêntica no treino e na inferência, eliminando os falsos negativos de identidade do PC3, e uma classe negativa que reforça a fronteira contra intrusos. Os resultados demonstram um protótipo estável, sem as falhas dos estágios anteriores: identidade consistente sob variação de iluminação, rejeição de intrusos e de ataques de apresentação, e comunicação entre processos sem perda de bytes.

I. INTRODUÇÃO

A segurança residencial demanda autonomia e privacidade. Diferente de soluções comerciais como Ring [1] ou Nest [2], que dependem de servidores externos, este projeto realiza todo o processamento localmente (*Edge Computing*), mantendo o dado biométrico dentro do dispositivo. O sistema implementa uma portaria eletrônica com reconhecimento facial, detecção de vivacidade (*liveness*) e acionamento de uma trava solenoide.

O PC4 é o **refinamento final** do protótipo, com quatro objetivos: (i) migrar todo o código de Python para C/C++; (ii) adicionar recursos de tempo real do Linux ao núcleo de controle [9], [10]; (iii) corrigir os falsos negativos de identidade observados no PC3 sob variação de iluminação; e (iv) tornar o classificador robusto contra intrusos por meio de uma classe negativa. As seções seguintes descrevem o hardware (II-A), a arquitetura de software em C++ (II-B), o processo de calibração e os resultados (III).

II. DESENVOLVIMENTO DO PROJETO

A. Descrição de Hardware

O sistema utiliza uma Raspberry Pi 3 Model B como núcleo. A lógica de controle e a sinalização (*driver* e LED RGB) são alimentadas pelo barramento de 5 V da própria Pi, enquanto a trava solenoide é acionada por uma fonte de 12 V. As duas

fontes compartilham um **barramento de terra comum** — ponto cuja garantia corrige a anomalia elétrica (LED fantasma) observada no PC3. A Tabela I lista os materiais.

Tabela I
LISTA DE MATERIAIS (BOM) – IMPLEMENTAÇÃO REAL (PC4)

Componente	Quantidade
Raspberry Pi 3 Model B	1
Trava Solenoide 12 V	1
Fonte de Alimentação 12 V	1
Transistor TIP31C (NPN, potência)	1
Transistor BC548 (NPN, driver)	1
Diodo Retificador 1N4007 (<i>flyback</i>)	1
Resistor 330 Ω	4
Resistor 10 k Ω	1
LED RGB de cátodo comum	1
Capacitor Eletrolítico 470 μ F / 20 V	1
Webcam USB	1

1) *Diagrama de Blocos*: O diagrama de blocos da Fig. 1 mostra os componentes de hardware. Os fluxos de **sinal/dados** aparecem como setas finas e os de **energia/potência** como setas grossas duplas. A lógica é alimentada pelo barramento de 5 V da Raspberry Pi e a solenoide por uma fonte de 12 V, ambas com terra comum.

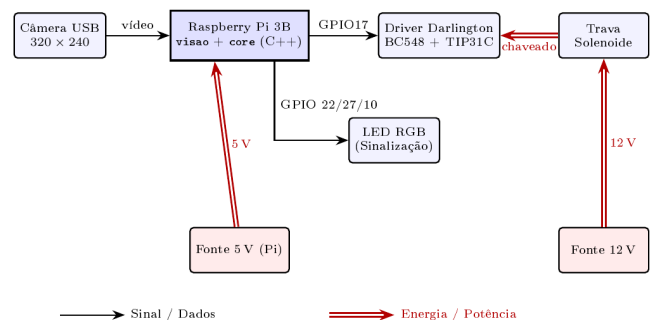


Figura 1. Diagrama de blocos do sistema: componentes de hardware, com distinção entre fluxos de sinal (setas finas) e de energia (setas grossas).

2) *Estágio de Potência e Esquemático*: O acionamento da solenoide usa um par Darlington (BC548+TIP31C) como chave de baixo lado. A GPIO 17 controla a base do BC548 através de um resistor de 330 Ω ; um resistor de *pull-down*

de 10 k Ω garante o estado desligado na ausência de sinal. O coletor do BC548 liga-se ao 5 V e seu emissor à base do TIP31C, cujo coletor chaveia a solenoide e cujo emissor vai ao terra. A solenoide é alimentada pelo **barramento de 12 V** (terminal positivo no +12 V), e um diodo *flyback* 1N4007 fica em antiparalelo com a bobina (cátodo, lado da listra cinza, no +12 V) para suprimir o pico de tensão reversa no desligamento. Um capacitor eletrolítico de 470 μ F/20 V filtra o barramento de 5 V. O LED RGB de cátodo comum é acionado pelas GPIOs 22/27/10, cada perna com um resistor de 330 Ω . O esquemático completo é apresentado na Fig. 2.

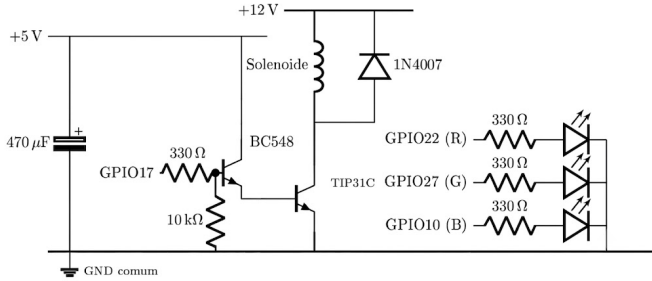


Figura 2. Esquema elétrico: par Darlington BC548+TIP31C acionando a solenoide a partir do barramento de 12 V, com diodo de roda-livre 1N4007, capacitor de filtro de 470 μ F e LED RGB de sinalização.

B. Descrição de Software

No PC4, todos os *scripts* Python do PC3 foram reescritos em C++. O sistema permanece como **dois processos independentes** sincronizados por um *pipe* nomeado (FIFO) em /tmp/gate_pipe: o processo visao envia o veredito ('F' = autorizado, 'I' = intruso) e o sistema_core aciona o hardware. A Tabela II resume a migração.

Tabela II
MIGRAÇÃO DE PYTHON (PC3) PARA C++ (PC4).

PC3 (Python)	PC4 (C++)	Função
vision.py	visao.cpp	Pipeline de visão
treinar_e_ver.py	treinar.cpp	Treino do classificador
treinar_increment.py	(no treinar.cpp)	Absorvido
main.cpp	core.cpp	Núcleo + tempo real
—	visao_core.hpp	Algoritmos compartilhados
—	extrair rostos.cpp	Extração de rostos

1) *Núcleo de Controle e Recursos de Tempo Real*: O core.cpp controla exclusivamente o hardware por meio de duas *threads* (std::thread): threadMonitorPipe, que escuta o FIFO com poll() (sem busy-wait), e threadControleLED, que atualiza a sinalização. O estado compartilhado é protegido por std::mutex (mtx para o estado e hwMtx para serializar o acesso às GPIOs), e os sinais SIGINT/SIGTERM garantem encerramento limpo. Sobre essa base, o PC4 adiciona quatro recursos de tempo real do Linux:

- [RT1] mlockall(MCL_CURRENT | MCL_FUTURE) — trava toda a memória do processo na RAM, impedindo

que páginas sejam enviadas ao swap em um momento crítico (Listagem 1).

- [RT2] SCHED_FIFO — a thread crítica recebe escalonamento de tempo real com prioridade fixa (50), executando imediatamente quando o byte chega, sem ser preemptada por processos comuns.
- [RT3] Afinidade de CPU — cada thread é fixada em um núcleo (monitor na CPU 3, LED na CPU 2), evitando migração entre núcleos e o jitter associado à invalidação de cache (Listagem 2).
- [RT4] Tarefa periódica absoluta — a thread do LED usa clock_nanosleep com CLOCK_MONOTONIC e TIMER_ABSTIME: o próximo instante é sempre “anterior + 100 ms”, sem o drift acumulado do sleep_for comum (Listagem 3).

```
if (mlockall(MCL_CURRENT | MCL_FUTURE) == 0)
    std::cout << "[RT] Memória travada na RAM.\n";
```

Listing 1. Travamento de memória na RAM (core.cpp).

```
bool configurarTempoReal(int prioridade, int cpu, const
    char* nome) {
    cpu_set_t set; CPU_ZERO(&set); CPU_SET(cpu, &set);
    pthread_setaffinity_np(pthread_self(), sizeof(set), &
        set); // [RT3]
    struct sched_param sp; std::memset(&sp, 0, sizeof(sp));
    sp.sched_priority = prioridade;
    // [RT2]
    return pthread_setschedparam(pthread_self(), SCHED_FIFO,
        &sp) == 0;
}
```

Listing 2. SCHED_FIFO e afinidade de CPU (core.cpp).

```
const long PERIODO_NS = 100L*1000L*1000L; // 100 ms
struct timespec proximo; clock_gettime(CLOCK_MONOTONIC, &
    proximo);
while (rodando) {
    /* ... atualiza LED conforme estado ... */
    proximo.tv_nsec += PERIODO_NS;
    while (proximo.tv_nsec >= 1000000000L) { proximo.
        tv_nsec -= 1000000000L; proximo.tv_sec++; }
    clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &
        proximo, nullptr);
}
```

Listing 3. Período exato com clock_nanosleep absoluto (core.cpp).

Tabela III
RECURSOS DE SISTEMA OPERACIONAL EMPREGADOS NO PC4.

Recurso	Chamada de sistema
Múltiplos processos	visao + sistema_core
Threads	std::thread
Pipe nomeado (IPC)	mkfifo/open/read/write
Sinais	signal(SIGINT/SIGTERM)
Exclusão mútua	std::mutex (mtx, hwMtx)
E/S eficiente	poll() com timeout
Memória travada [RT1]	mlockall()
Escalonamento RT [RT2]	SCHED_FIFO
Afinidade de CPU [RT3]	pthread_setaffinity_np()
Tarefa periódica [RT4]	clock_nanosleep(ABSTIME)

2) *Algoritmos Compartilhados (visao_core.hpp)*: Todo o “coração matemático” do sistema fica no header visao_core.hpp, garantindo que treino e inferência processem as imagens de forma *idêntica*. Como o módulo

`cv::face` pertence ao `opencv_contrib` (que não vem no pacote padrão da Pi e leva horas para compilar), o classificador **LBPH** foi reimplementado do zero, dependendo apenas do OpenCV base. O *header* reúne: `tanTriggs()` (normalização de iluminação), `preprocessarFace()` (recorte, redimensionamento para 120×120 e Tan-Triggs), `computeLBP()`, `spatialHistogram()` (grade 8×8 , 256 *bins* por célula, vetor de 16.384 *floats*), `chiSquare()` (distância χ^2), a struct `LBPHModel` (treino, predict, persistência em texto) e `fftRatio()` (razão de energia espectral para *liveness*).

3) *Correção dos Falsos Negativos: Normalização Tan-Triggs*: No PC3, o LBPH retornava distâncias de 84 a 98 para o usuário legítimo sob iluminação diferente da do treino, rejeitando-o. A causa raiz era um **pipeline assimétrico**: o treino usava a ROI bruta, enquanto a inferência aplicava CLAHE apenas na detecção — o modelo aprendia uma textura e tentava reconhecer outra. A correção foi a normalização **Tan-Triggs** [8], aplicada de forma idêntica nos dois lados via `preprocessarFace()` (Listagem 4). Ela combina correção *gamma*, diferença de Gaussianas (remove a iluminação de baixa frequência) e equalização de contraste com saturação em *tanh*. Resultado: o mesmo rosto sob luz forte ou fraca gera imagens processadas semelhantes, e a distância do usuário legítimo caiu para a faixa **34–38**, estável.

```
inline cv::Mat preprocessarFace(const cv::Mat& grayFrame,
    const cv::Rect& r) {
    cv::Rect rr = r & cv::Rect(0,0,grayFrame.cols,grayFrame.rows);
    if (rr.width < 40 || rr.height < 40) return cv::Mat();
    cv::Mat roi = grayFrame(rr), resized;
    cv::resize(roi, resized, cv::Size(FACE_SIZE, FACE_SIZE),
        0,0, cv::INTER_AREA);
    return tanTriggs(resized); // mesma normalizacao dos
        dois lados
}
```

Listing 4. Pré-processamento canônico, idêntico no treino e na inferência (`visao_core.hpp`).

4) *Classe Negativa*: No PC3 o classificador era treinado apenas com o usuário (rótulo 1); qualquer desconhecido era forçado a ser o rótulo 1. No PC4 criou-se a pasta `dataset_outros/` com rostos de terceiros (rótulo 2), extraídos com a ferramenta `extrair rostos.cpp` (que detecta todos os rostos de imagens em grade e salva recortes 120×120). O `treinar.cpp` lê as duas pastas e aprende a **fronteira** entre “Fernando” (1) e “desconhecido” (2). Como o acesso só é liberado quando `idUsuario == 1`, o intruso classificado como rótulo 2 é barrado automaticamente, independente da distância χ^2 .

5) *Pipeline de Visão* (`visao.cpp`): Para cada quadro 320×240 : conversão para cinza, CLAHE (apenas na detecção), detecção por Haar Cascades, `preprocessarFace()`, `predict()` (rótulo + distância) e quatro métricas de *liveness* (LBP, Laplaciano, Sobel e FFT). Um quadro é REAL se o *score* ≥ 3 e LBP e FFT passarem. Os vereditos entram em janelas deslizantes (deque de 10); quando $\geq 6/10$ quadros são reais e $\geq 6/10$ são do usuário autorizado (rótulo 1, *Conf* < 38), envia-se ‘F’; se menos de 4 quadros passarem, envia-se ‘I’ e uma foto ao Telegram. O modo `-calibrar` executa

o mesmo *pipeline* sem disparar ações, para medir os limiares. O fluxograma é apresentado na Fig. 3.

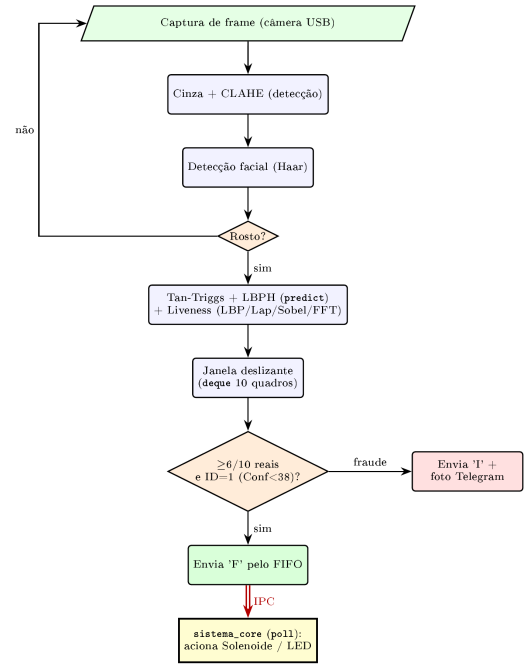


Figura 3. Fluxograma do *pipeline* de visão: da captura ao acionamento do hardware via IPC.

III. CALIBRAÇÃO E RESULTADOS EXPERIMENTAIS

A. Processo de Calibração

A calibração foi iterativa, com pessoas reais. Inicialmente, com `CONF_MAX=55`, um amigo do usuário conseguia abrir o sistema (distâncias entre 36 e 48, sobrepondo-se às do usuário). A primeira iteração reduziu o limiar para `CONF_MAX=38`, o que melhorou mas não resolveu (o amigo ainda descia a 35–37 em certas posições). A segunda iteração adicionou a **classe negativa** (63 imagens de terceiros), após a qual o amigo passou a ser classificado como rótulo 2 na maioria dos quadros, sendo barrado pelo critério `idUsuario == 1`. Os limiares finais são: `CONF_MAX=38`, `LBP_MIN=3,7`, `LAP_MIN=28`, `GRAD_STD_MIN=12` e `FFT_RATIO=0,65`.

B. Resultados

A Listagem 5 mostra um log típico do usuário legítimo.

```

ID=1(Conf:35.4) | LBP=4.30 LAP=121.40 GRD=18.70 FFT=0.830
[4/4] -> [REAL]
ID=1(Conf:36.8) | LBP=4.20 LAP=110.70 GRD=17.90 FFT=0.790
[4/4] -> [REAL]
[ACESSO CONCEDIDO] Identidade e liveness confirmados.
[CORE] Acesso autorizado. Abrindo trava.
  
```

Listing 5. Log de execução: reconhecimento e *liveness* do usuário legítimo.

Os principais resultados são:

- **Identidade**: usuário legítimo aceito de forma consistente, com *Conf* 34–38 estável sob variação de iluminação (era 84–98 no PC3).

- **Intruso:** após a classe negativa, classificado como rótulo 2 em 80–90% dos quadros; o sistema não abre.
- **Anti-spoofing:** foto em tela de celular resulta em rótulo 2 ou LBP baixo $\Rightarrow$ alerta de intruso, com notificação e foto no Telegram.
- **IPC:** estável, sem perda de bytes, graças ao descritor de leitura persistente com `poll()`.
- **Tempo real:** `mlockall`, `SCHED_FIFO` e afinidade de CPU bem-sucedidos (com `sudo`) nos quatro núcleos da Pi 3; o LED opera em período exato de 100 ms.
- **CPU:** consumo baixo, pela ausência de *busy-wait* (`poll()`) e pelo *sleep* de 150 ms entre quadros na visão.

C. Evoluções em Relação ao PC3

Quatro problemas do PC3 foram corrigidos: (i) **perda de bytes no IPC** — o lado escritor abria/fechava o FIFO a cada quadro, criando janelas sem leitor; resolvido mantendo o descritor de leitura persistente com `poll()`; (ii) **pipeline assimétrico** entre treino e inferência, resolvido por `preprocessarFace()` (Seção II-B); (iii) **CLAHE recriado a cada quadro**, agora instanciado uma única vez fora do laço; e (iv) a **anomalia elétrica do LED fantasma**, causada pelo rompimento do barramento de terra comum entre as fontes de 5 V e 12 V, eliminada com a troca dos *jumpers* e a garantia da continuidade desse terra comum.

IV. CONCLUSÕES

O PC4 consolidou o protótipo em um sistema integrado, inteiramente em C/C++, estável e sem as falhas dos estágios anteriores. A migração permitiu acoplar recursos de tempo real do Linux (memória travada, escalonamento `SCHED_FIFO`, afinidade de CPU e tarefa periódica absoluta) que garantem latência determinística no acionamento da trava. As duas correções biométricas centrais — a normalização Tan-Triggs idêntica entre treino e inferência, e a introdução de uma classe negativa — resolveram, respectivamente, os falsos negativos de identidade e a fragilidade do classificador de classe única.

Entre os aprendizados: a robustez de um sistema embarcado depende tanto do software quanto da integridade física (a anomalia de terra do PC3 reforçou a atenção à continuidade do barramento de terra comum entre as fontes de 5 V e 12 V); e a coerência do *pipeline* de pré-processamento entre treino e inferência é tão importante quanto a escolha do classificador. Como trabalhos futuros, propõe-se a substituição do LBPH por *embeddings* faciais de rede neural (mais tolerantes a pose e iluminação) e a implementação da senha temporária para entregadores prevista desde o PC1.

AGRADECIMENTOS

O autor agradece ao professor e aos colegas da disciplina de Sistemas Operacionais Embarcados pelas sugestões ao longo dos pontos de controle, que orientaram o refinamento do protótipo.

REFERÊNCIAS

- [1] Ring LLC, “Ring video doorbell,” 2024. [Online]. Available: <https://ring.com/>
- [2] Google Store, “Google nest doorbell,” 2024. [Online]. Available: https://store.google.com/product/nest_doorbell
- [3] Itseez, “Open source computer vision library,” 2015. [Online]. Available: <https://opencv.org>
- [4] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” *CVPR*, 2001.
- [5] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” *Graphics Gems IV*, 1994.
- [6] T. Ojala, M. Pietikainen, and T. Maenpää, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE PAMI*, vol. 24, no. 7, pp. 971–987, 2002.
- [7] T. Ahonen, A. Hadid, and M. Pietikainen, “Face description with local binary patterns: Application to face recognition,” *IEEE PAMI*, vol. 28, no. 12, pp. 2037–2041, 2006.
- [8] X. Tan and B. Triggs, “Enhanced local texture feature sets for face recognition under difficult lighting conditions,” *IEEE Trans. Image Processing*, vol. 19, no. 6, pp. 1635–1650, 2010.
- [9] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.
- [10] M. Kerrisk, *The Linux Programming Interface*. No Starch Press, 2010.

ANEXO I – ACESSO AO REPOSITÓRIO DO PROJETO

O código-fonte integral (todo em C/C++), o histórico de versões e os arquivos de configuração estão hospedados em ambiente Git público.

O repositório contém:

- **core.cpp:** núcleo de controle com recursos de tempo real;
- **visao.cpp:** *pipeline* de visão (*liveness* + reconhecimento);
- **visao_core.hpp:** algoritmos compartilhados (Tan-Triggs, LBPH, FFT);
- **treinar.cpp:** treino do classificador (multi-ângulo e por datasets);
- **extrair rostos.cpp:** extração de rostos para a classe negativa;
- **Makefile e haarcascade_frontalface_default.xml.**

Observação: o modelo `modelo_lbph.txt` e as pastas `dataset_fernando/` e `dataset_outros/` (dados biométricos) são gerados localmente e não são versionados.

Link de Acesso: <https://github.com/FuyteorUnB/SOE-2026.1/tree/PC4>